Overview

Data structures and algorithms determine latency, memory cost, and scalability. AI systems process large streams of events, vectors, and metadata; wrong structure choice can dominate runtime even when model code is efficient.

Core Data Structures

Arrays & Lists

- **Array (concept):** contiguous memory sequence of homogeneous types.
- **Python list:** dynamic array-like container of object references.

List indexing is $O(1)$ average; insertion/removal near front is $O(n)$ due to shifts.

Tuples

Immutable ordered collection. Useful for fixed-size records and dictionary keys.

Sets

Unordered unique elements with average $O(1)$ membership checks. Useful for deduplication and fast "seen" tracking.

Hash Maps (Dictionaries)

Key-value mapping using hash table. Average $O(1)$ lookup/insert/delete. Fundamental for counters, caches, and join-like lookups.

Stacks & Queues

- Stack: LIFO discipline.
- Queue: FIFO discipline.

Use `list` for stack, `collections.deque` for queue.

Trees & Graphs (High-Level)

- Tree: acyclic hierarchy.
- Graph: nodes + edges representing arbitrary relations.

Many AI tasks are graph-like: dependency DAGs, recommendation graphs, workflow DAG execution.

Core Algorithms

Search

- Linear search: $O(n)$, no sort requirement.
- Binary search: $O(\log n)$, requires sorted data.

Sorting

- Insertion sort: $O(n^2)$ worst-case, good teaching tool.
- Timsort (`sorted`): optimized hybrid with typical $O(n \log n)$ behavior.

Graph Traversal

- DFS explores depth-first; memory-efficient in sparse shallow graphs.
- BFS explores layer-wise; useful for shortest path in unweighted graphs.

Big-O Complexity (Intuition)

Big-O describes asymptotic growth of runtime/space with input size n.

- $O(1)$ constant.
- $O(\log n)$ logarithmic.
- $O(n)$ linear.
- $O(n \log n)$ near-optimal sort/merge patterns.
- $O(n^2)$ quadratic; quickly becomes impractical.

Space complexity matters too. Algorithm with low runtime but high memory can crash containerized services.

Example Problems and Solutions

1. **Frequency counting:** dictionary-based linear pass.
2. **Shortest traversal depth:** BFS with queue.
3. **Cycle detection (directed graph):** DFS recursion stack or iterative state coloring.
4. **Top-k retrieval:** heap-based approach vs full sort tradeoff.

Business Case Studies & Exceptions

Case 1: Log Processing Pipeline

Naive nested loops for deduplication created $O(n^2)$ runtime and missed SLA at peak traffic.

Fix: - Replace list membership checks with set. - Use dictionary aggregation keyed by `user_id`.

Result: near-linear behavior and predictable latency.

Case 2: Recommendation Candidate Routing

Queue implemented with list `pop(0)` caused repeated shifts and CPU spikes.

Fix: - Switch to `deque.popleft()`. - Add guardrail on queue size and backpressure metrics.

Case 3: DFS Recursion Depth

Deep dependency graph triggered recursion depth errors in production.

Fix: - Use iterative DFS with explicit stack. - Add max-depth checks and alerting.

Interview Questions & Answers

1. **Q: List vs tuple?**
   A: List is mutable and dynamic; tuple is immutable and better for fixed records/hashing.
2. **Q: Why use dictionary over list for lookup-heavy workloads?**
   A: Dictionary average lookup is $O(1)$; list lookup is $O(n)$.
3. **Q: Binary search complexity and requirement?**
   A: $O(\log n)$ time, requires sorted input.

4. **Q: BFS vs DFS in one sentence?**
   A: BFS explores by levels and helps shortest path in unweighted graphs; DFS explores one branch deeply first.

5. **Q: Why does `pop(0)` on list hurt performance?**
   A: Elements shift left after removal, giving $O(n)$ per operation.

6. **Q: When is $O(n^2)$ acceptable?**
   A: Small bounded datasets or one-time offline preprocessing where simplicity wins.

7. **Q: What structure for streaming deduplication?**
   A: Set for seen keys, optionally with TTL window for memory control.

8. **Q: How does complexity affect cloud cost?**
   A: Higher complexity increases CPU time, latency, and infrastructure scaling requirements.

9. **Q: Give AI example of graph traversal.**
   A: Traversing feature dependency DAG or knowledge graph neighbors for candidate expansion.

10. **Q: Runtime vs space tradeoff example?**
    A: Caching intermediate results speeds repeated queries but increases memory usage.